

NASM Syntax Basics

- **Format:** `label: instruction operands ; comment`
- **Constants:**
 - Decimal: 200
 - Hex: 0xc8 or 0ach
 - Binary: 11001000b
- **Expressions:** NASM allows basic math in labels (e.g., `mov rax, label + 10`).
- **Case Sensitivity:** Instructions are case-insensitive, but labels are case-sensitive by default.

Data Movement: MOV and LEA

■ MOV instruction: The workhorse.

- `mov rax, 42` (Immediate to register)
- `mov [var], rax` (Register to memory)

■ The Bracket Rule: In NASM, `[address]` means "the contents of memory at this address."

- `mov rax, my_var` → rax gets the address of the variable.
- `mov rax, [my_var]` → rax gets the value stored there.

■ LEA (Load Effective Address): Calculates an address without accessing memory.

- `lea rdi, [rbx + rcx * 4]` (Commonly used for fast pointer math).

Defining Data (Directives)

■ Initialized (.data):

- db (byte), dw (word), dd (dword), dq (qword).
- Example: msg db 'Hello', 10, 0

■ Uninitialized (.bss):

- resb, resw, resd, resq (reserve N units).
- Example: buffer resb 64

■ Equates:

- STDOUT equ 1 — Constant definition (pre-processor).

Arithmetic and Logic

- **Basic Ops: add, sub, imul, idiv.**
- **Logic: and, or, xor, not.**
- **XOR Trick: xor rax, rax is the standard way to set a register to zero (way shorter and faster than mov rax, 0).**
- **The Flags Register (RFLAGS):**
 - ZF (Zero Flag), SF (Sign Flag), CF (Carry Flag).
 - Updated automatically by arithmetic instructions.

Control Flow

- **Unconditional: `jmp label`**
- **Conditional: Based on `CMP` (Compare) or `TEST`.**
 - `cmp rax, rbx`
 - `je label` (Jump if equal / `ZF=1`)
 - `jne label` (Jump if not equal)
 - `jg / jl` (Signed Jump Greater/Less)
 - `ja / jb` (Unsigned Jump Above/Below)
- **Loops: While NASM has a `loop` instruction, manual `dec` and `jnz` are usually preferred for performance.**

Some Arithmetic Operations

■ Two Operand Instructions:

<i>Format</i>	<i>Computation</i>	
<code>addl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} + \text{Src}$
<code>subl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} - \text{Src}$
<code>imull</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} * \text{Src}$
<code>sall</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \ll \text{Src}$
<code>sarl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \gg \text{Src}$
<code>shrl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \gg \text{Src}$
<code>xorl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \wedge \text{Src}$
<code>andl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \& \text{Src}$
<code>orl</code>	<i>Src, Dest</i>	$\text{Dest} = \text{Dest} \text{Src}$

Also called `shll`

Arithmetic

Logical

■ Watch out for argument order!

■ No distinction between signed and unsigned int (why?)

Some Arithmetic Operations

■ One Operand Instructions

`incl` *Dest* $Dest = Dest + 1$

`decl` *Dest* $Dest = Dest - 1$

`negl` *Dest* $Dest = -Dest$

`notl` *Dest* $Dest = \sim Dest$

■ See book for more instructions

Arithmetic Expression Example

```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```

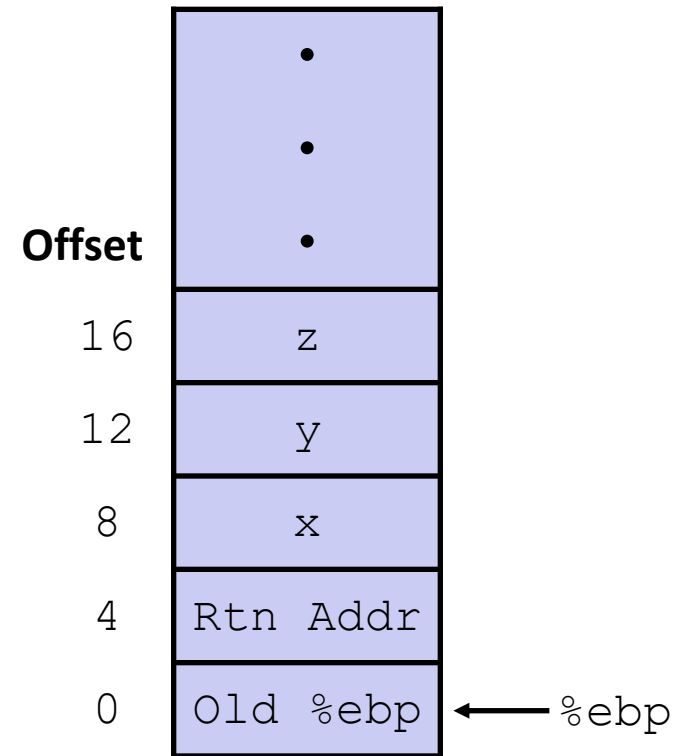
arith:

pushl	%ebp	} Set Up
movl	%esp, %ebp	
movl	8(%ebp), %ecx	} Body
movl	12(%ebp), %edx	
leal	(%edx,%edx,2), %eax	
sall	\$4, %eax	
leal	4(%ecx,%eax), %eax	
addl	%ecx, %edx	
addl	16(%ebp), %edx	
imull	%edx, %eax	
popl	%ebp	} Finish
ret		

Understanding arith

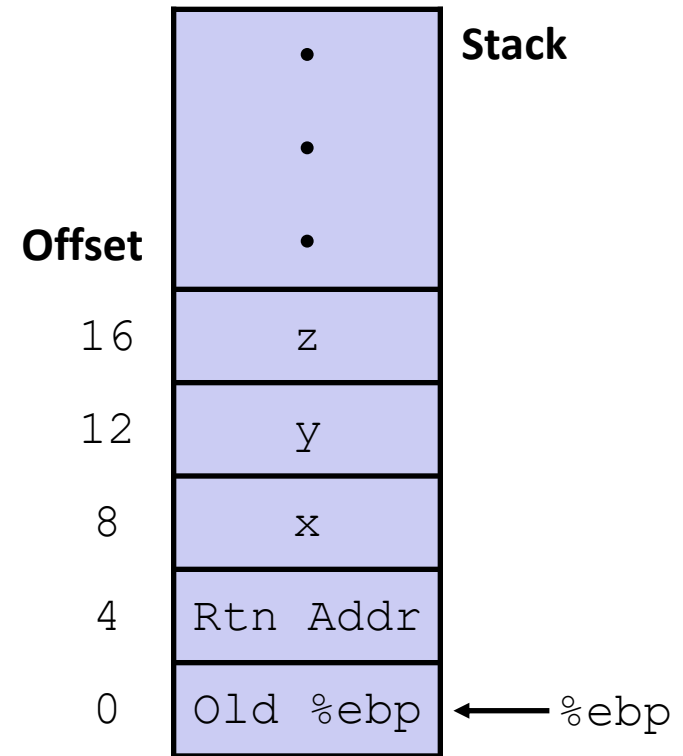
```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```

```
movl    8(%ebp), %ecx
movl    12(%ebp), %edx
leal    (%edx,%edx,2), %eax
sall    $4, %eax
leal    4(%ecx,%eax), %eax
addl    %ecx, %edx
addl    16(%ebp), %edx
imull   %edx, %eax
```



Understanding arith

```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```



movl	8(%ebp), %ecx	# ecx = x
movl	12(%ebp), %edx	# edx = y
leal	(%edx,%edx,2), %eax	# eax = y*3
sall	\$4, %eax	# eax *= 16 (t4)
leal	4(%ecx,%eax), %eax	# eax = t4 + x + 4 (t5)
addl	%ecx, %edx	# edx = x + y (t1)
addl	16(%ebp), %edx	# edx += z (t2)
imull	%edx, %eax	# eax = t2 * t5 (rval)

Observations about `arith`

```
int arith(int x, int y, int z)
{
    int t1 = x+y;
    int t2 = z+t1;
    int t3 = x+4;
    int t4 = y * 48;
    int t5 = t3 + t4;
    int rval = t2 * t5;
    return rval;
}
```

- Instructions in different order from C code
- Some expressions require multiple instructions
- Some instructions cover multiple expressions
- Get exact same code when compile:
 - $(x+y+z) * (x+4+48*y)$

<code>movl</code>	<code>8(%ebp), %ecx</code>	<code># ecx = x</code>
<code>movl</code>	<code>12(%ebp), %edx</code>	<code># edx = y</code>
<code>leal</code>	<code>(%edx,%edx,2), %eax</code>	<code># eax = y*3</code>
<code>sall</code>	<code>\$4, %eax</code>	<code># eax *= 16 (t4)</code>
<code>leal</code>	<code>4(%ecx,%eax), %eax</code>	<code># eax = t4 +x+4 (t5)</code>
<code>addl</code>	<code>%ecx, %edx</code>	<code># edx = x+y (t1)</code>
<code>addl</code>	<code>16(%ebp), %edx</code>	<code># edx += z (t2)</code>
<code>imull</code>	<code>%edx, %eax</code>	<code># eax = t2 * t5 (rval)</code>

Another Example

```
int logical(int x, int y)
{
    int t1 = x^y;
    int t2 = t1 >> 17;
    int mask = (1<<13) - 7;
    int rval = t2 & mask;
    return rval;
}
```

logical:

```
    pushl %ebp
    movl %esp,%ebp          } Set Up

    movl 12(%ebp),%eax
    xorl 8(%ebp),%eax
    sarl $17,%eax
    andl $8185,%eax        } Body

    popl %ebp
    ret                     } Finish
```

<code>movl 12(%ebp),%eax</code>	<code># eax = y</code>
<code>xorl 8(%ebp),%eax</code>	<code># eax = x^y (t1)</code>
<code>sarl \$17,%eax</code>	<code># eax = t1>>17 (t2)</code>
<code>andl \$8185,%eax</code>	<code># eax = t2 & mask (rval)</code>

Another Example

```
int logical(int x, int y)
{
    int t1 = x^y;
    int t2 = t1 >> 17;
    int mask = (1<<13) - 7;
    int rval = t2 & mask;
    return rval;
}
```

logical:

```
pushl %ebp
movl %esp,%ebp
```

} Set
Up

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

} Body

```
popl %ebp
ret
```

} Finish

```
movl 12(%ebp),%eax
xorl 8(%ebp),%eax
sarl $17,%eax
andl $8185,%eax
```

```
# eax = y
# eax = x^y      (t1)
# eax = t1>>17   (t2)
# eax = t2 & mask (rval)
```